

DSE6212

Text and Image Mining Session # 4

Agarwal



MERRIMACK COLLEGE

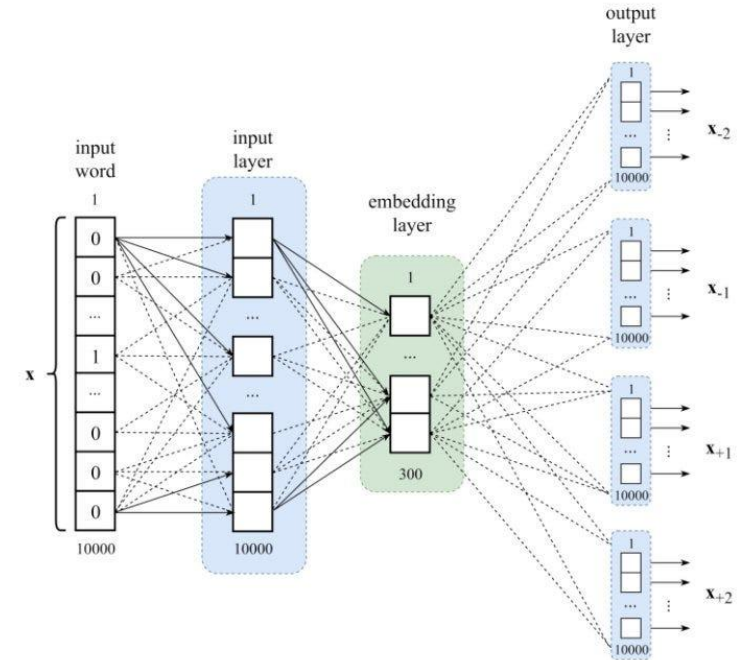
Neural Networks

Artificial Neural Net in NLP: Limitations

Traditional ANNs (Feedforward Networks) face two critical bottlenecks when processing natural language:

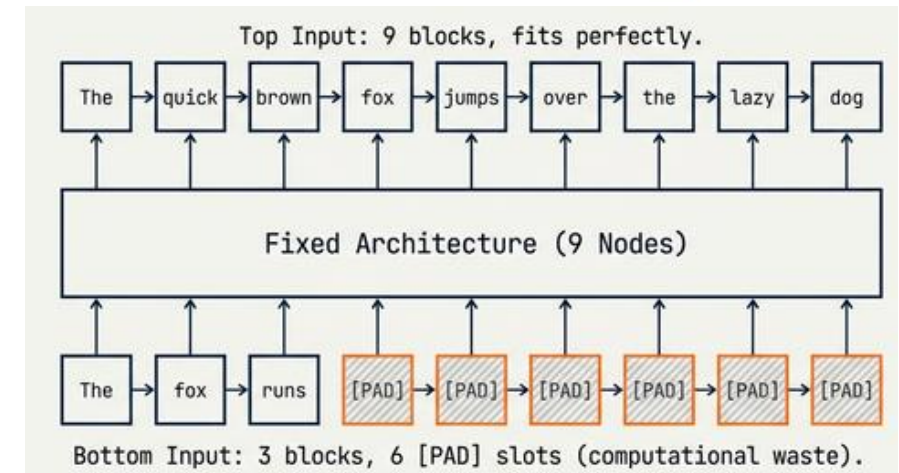
Loss of Sequence

- Standard word embeddings such as Word2Vec or GloVe, transform text into dense numerical representations. While they capture semantic relationships, they ignore the **sequential nature of language**
- Example:** In the sentence "The quick brown fox jumps over the lazy dog," a traditional feedforward network treats "fox," "jumps," and "dog" as separate entities, disregarding the subject-action-object relationship
- This is because Standard ANNs process all inputs in parallel. When you feed all words into a network simultaneously (e.g., using a Bag-of-Words approach), the model loses information about which word came first or last. It fails to capture the "semantic meaning" derived specifically from the sequence



Variable Length

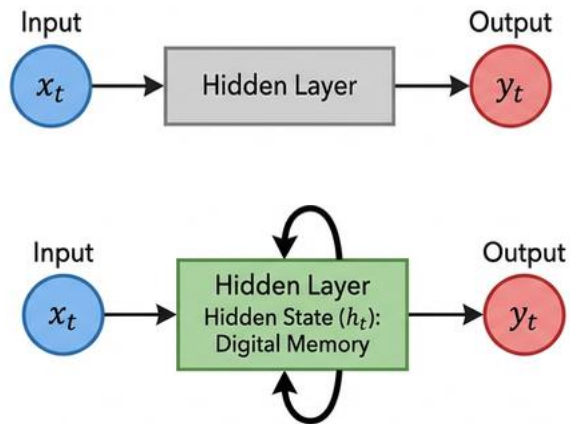
- Real-world text consists of sentences with **varying lengths** (e.g., "The fox runs" vs. "The quick brown fox jumps...")
- ANNs require a fixed input size (e.g., 100 input nodes). If your sentence is 5 words, you must **pad the rest** with zeros to meet the 100-node length
- This padding is a workaround that results in **sparse data** and does not solve the disconnection between words like "jumps" and "dog"



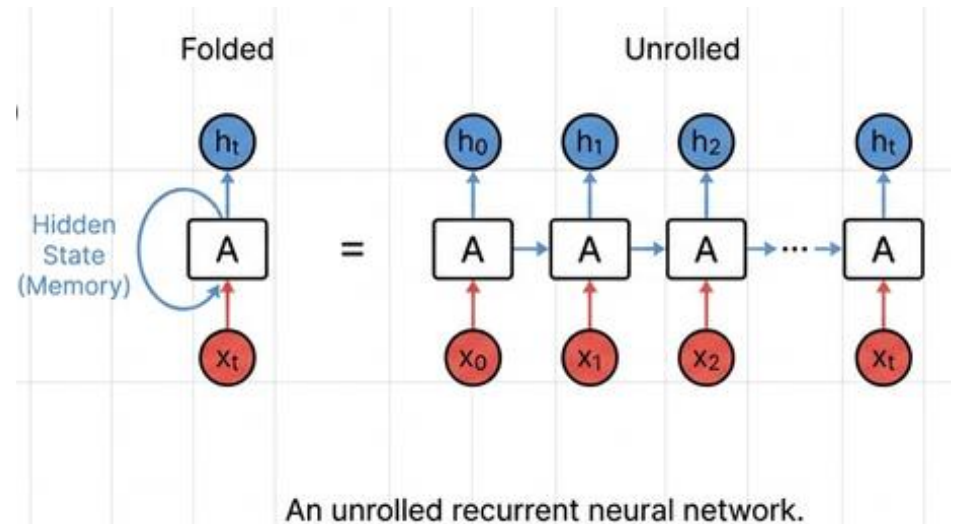
Deep Neural Net: RNN

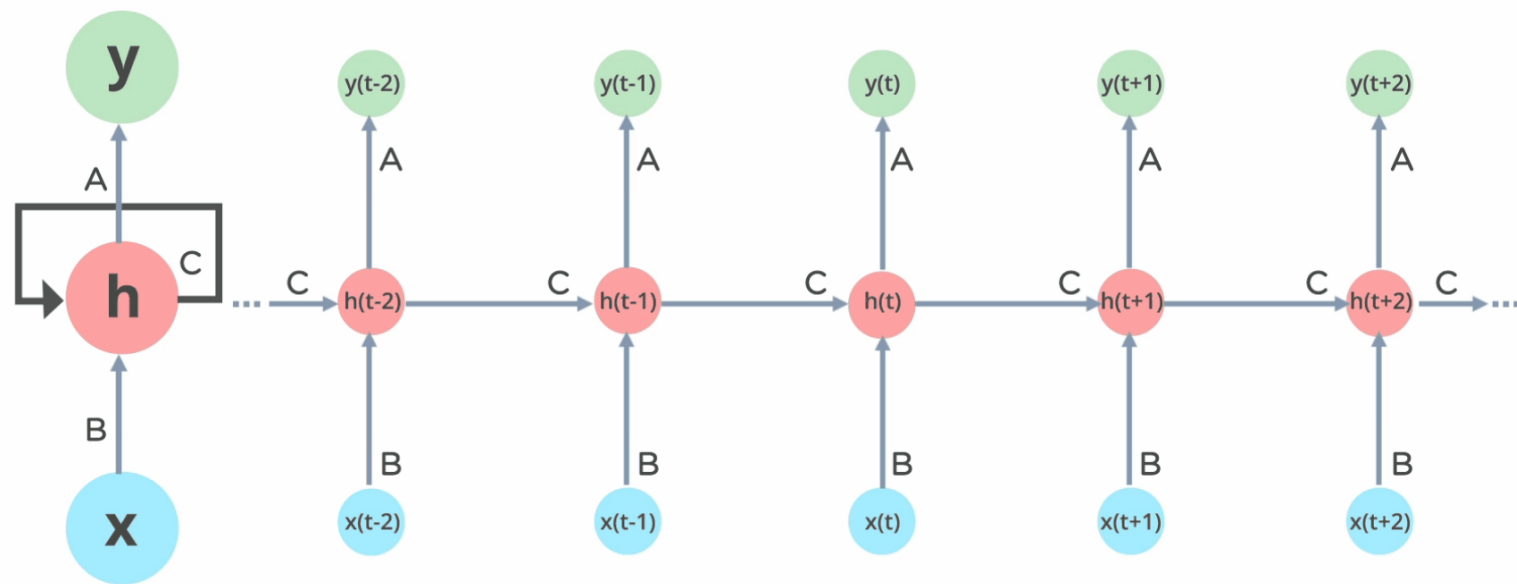
We need a network that reads the sentence like a human does: word by word, remembering what it just read.

- Recurrent Neural Networks (RNNs) are a specialized class of neural networks designed to handle sequential data.
- Unlike feedforward networks, RNNs have an internal loop where they feed their output back into themselves, thereby persisting context from previous steps, which acts as a **short-term memory**. This state is known as “**Hidden State**” (h_t)



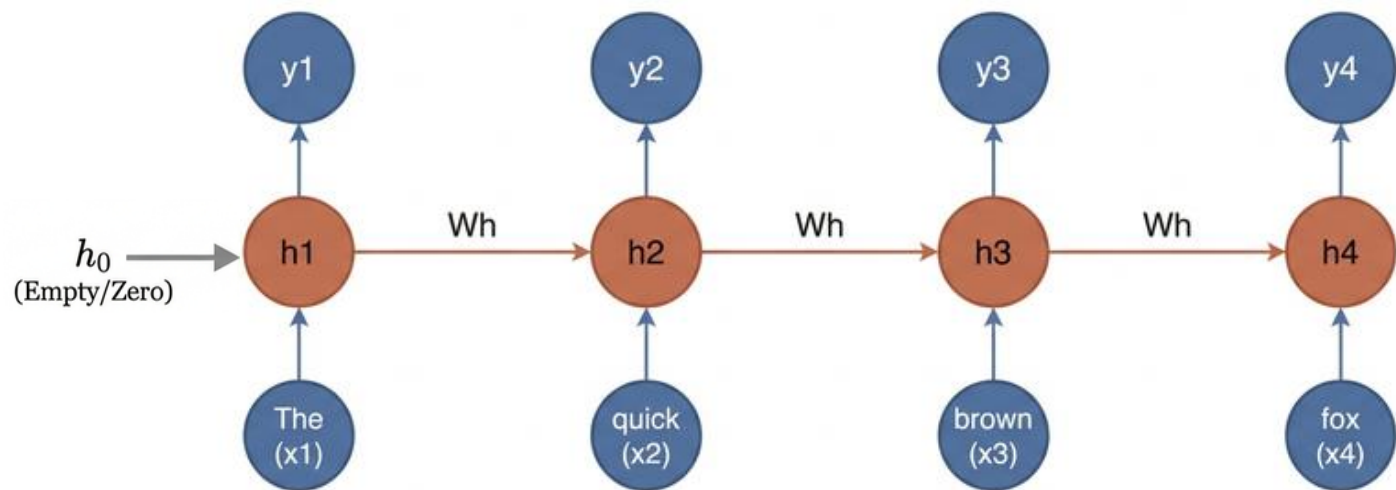
This allows the network to handle variable lengths naturally. It reads “The” remembers it, then reads “fox”, updating its understanding based on both.





RNN: Architecture

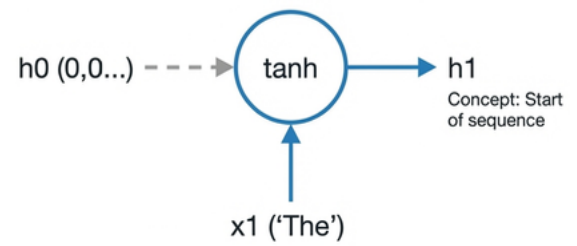
Let's break this down with an example. We will trace the sentence "The quick brown fox..." through the architecture



Before the first word is read; the network has no context. We must initialize the Hidden State as a vector of zeros.

h_0 (The Blank Slate) $\rightarrow \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \end{bmatrix} \quad h_0 = [0, 0, \dots, 0]$

Step 1: Processing "The" (t=1)

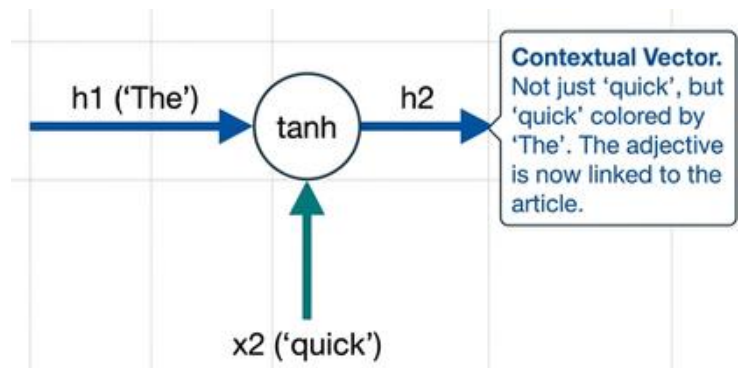


$$h_1 = \tanh(W_h \cdot h_0 + W_x \cdot x_1 + b)$$

Input (x1): At time Step 1, the word "The" is converted into its embedding vector. And becomes the first hidden state (h_1). Since there is no previous the network combines the empty memory (h_0) with the new word (x_1).

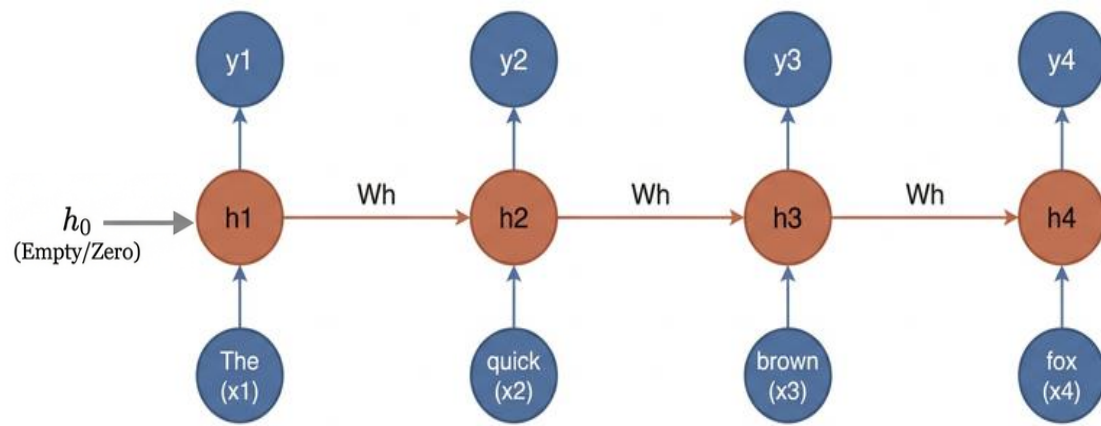
RNN: Architecture

Step 2: Processing “quick” (t=2)

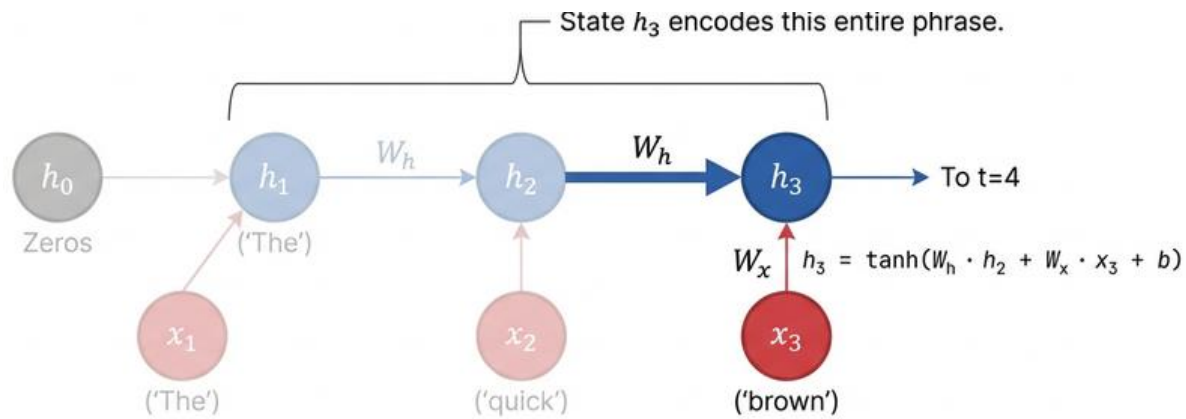


$$h_2 = \tanh(W_h \cdot h_1 + W_x \cdot x_2 + b)$$

Input (x_2): At time Step 2, we input the next word, “quick”. The network doesn’t just see “quick” in isolation; it receives the memory from Step 1. **The network now understands “The quick”**



Step 3: Deepening the Context “brown” (t=3)



The process repeats. By the end of the sentence, the final hidden state contains a compressed vector representation of the full story. This vector can now be used to predict the next word or classify sentiment

RNN: Mathematics of Short-Term Memory

Diagram illustrating the mathematical formula for the current hidden state h_t in an RNN:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

Labels and components:

- Previous Hidden State (Past Context):** h_{t-1}
- Activation Function (Squashes values between -1 and 1):** $\tanh$
- New Hidden State (Current Memory):** h_t
- Current Input (New Word):** x_t
- Shared Weights (The "Knowledge" of the Network):** W_h and W_x



Input Vector. The mathematical representation of the current word.



Previous Hidden State. The memory vector from the step before.



Current Hidden State. The new memory we are calculating now.

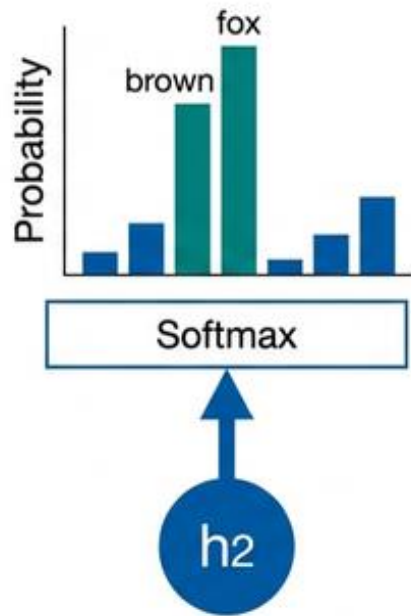


Weight Matrices. The learned parameters.

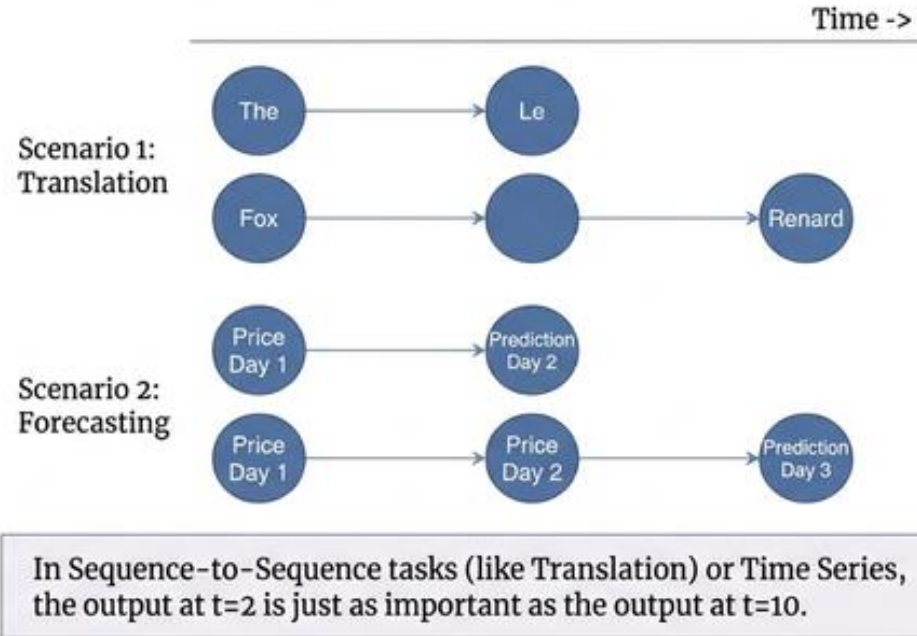
- W_x : Weights for new input.
- W_h : Weights for past memory.

RNN: Extracting Value from Intermediate Steps

We don't always wait for the end of the sentence. Intermediate outputs are vital for specific tasks. At any step, we can "tap into" the hidden state



$$y_2 = \text{softmax}(W_{hy} \mathbf{h}_2 + \mathbf{c})$$

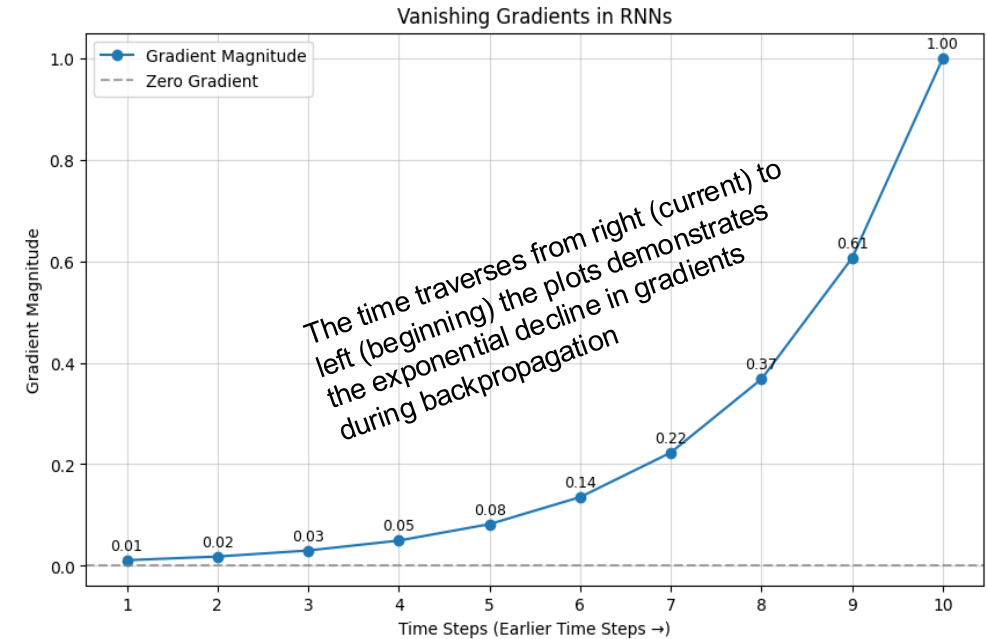


RNN: Challenge

The Vanishing Gradient Problem

While RNNs are designed to maintain memory through the hidden state, they suffer from a fundamental mathematical flaw: they are **forgetful** over long sequences. This is known as the **Vanishing Gradient Problem**.

To train an RNN, we calculate the gradient of the loss $\mathcal{L}$ with respect to the weights. Because the weights are shared across time, the gradient at time T must be propagated all the way back to time $t=1$. When we backpropagate through many time steps (e.g., a sentence with 100 words), we repeatedly multiply gradients. If the gradients are small (e.g., < 1), multiplying them repeatedly makes them tiny (e.g., $0.5 \times 0.5 \times 0.5 \dots \rightarrow 0$)



We calculate the gradient of the loss. Because the hidden state h_t depends on h_{t-1} , which depends on h_{t-2} , we must apply the **Chain Rule** backward all the way to the start of the sequence

$$\frac{\partial L_T}{\partial h_0} = \frac{\partial L_T}{\partial h_T} \times \frac{\partial h_T}{\partial h_{T-1}} \times \frac{\partial h_{T-1}}{\partial h_{T-2}} \times \dots \times \frac{\partial h_1}{\partial h_0}$$

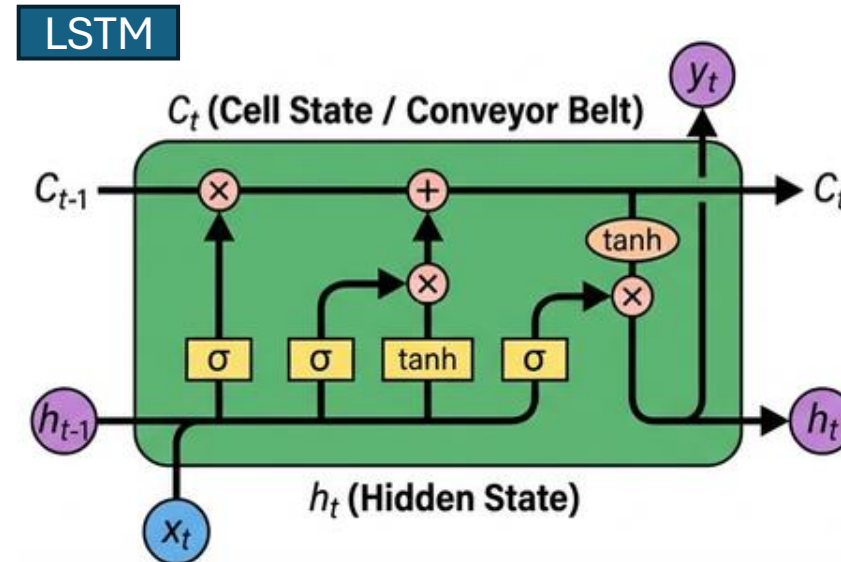
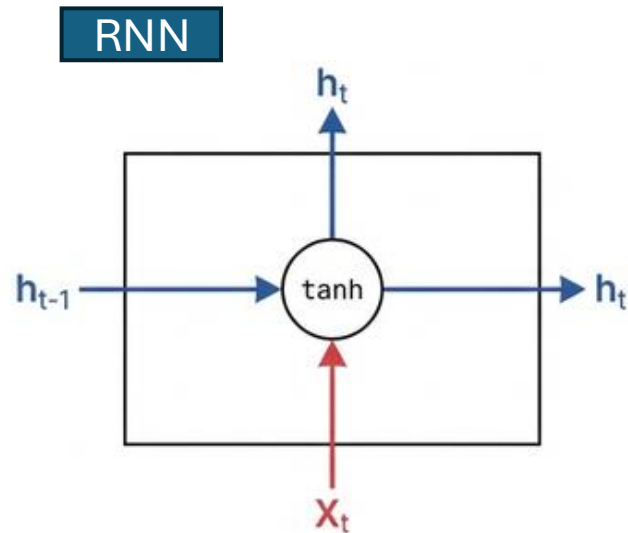
The network stops learning from the beginning of the sequence and becomes **short-sighted**. Ideally, in the sentence "**The clouds** [that were gathering over the dark, jagged peaks of the northern mountain range during the early hours of the morning] **were** grey". The network should **remember** "The clouds" with intermediate 20+ words of descriptive filler to predict the verb "**were**". But due to vanishing gradients, the "Plural" information from $t=1$ cannot travel through chain of 24 multiplication to reach $t=25$ and model loses connection between 'clouds' and 'were' defaulting to most recent noun seen ('morning was'). **The most recent inputs dominate the prediction**

Long Short-term Memory: LSTM

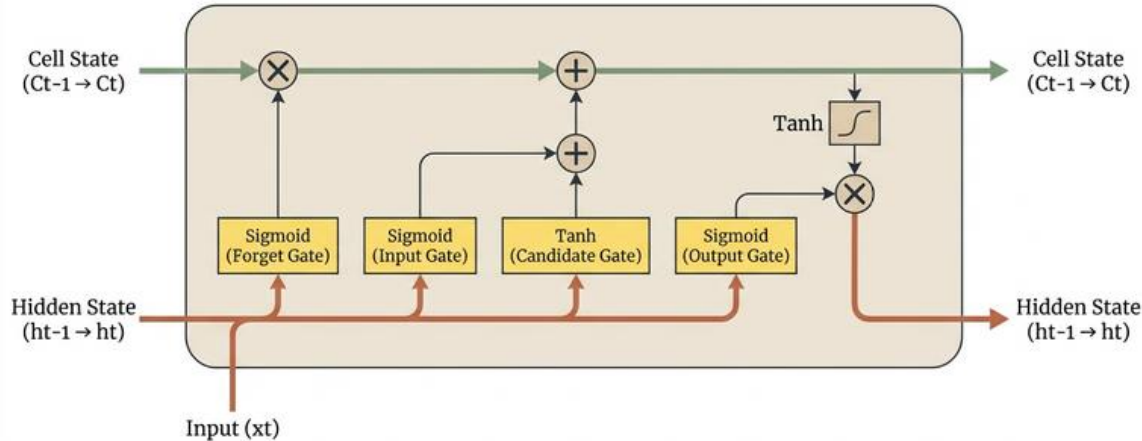
RNNs suffer from structural bottleneck where it's forced to perform **two conflicting roles** simultaneously: 1) working memory which represents the **immediate context** required for the current output and 2) carrying information from **distant past** to influence the far-off future

LSTMs resolve this by introducing two separate memory states using 'Gates' to manage information flow:

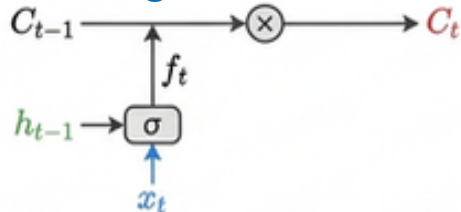
- **Cell State (C_t)**: This is the **long-term memory** that runs through the entire sequence with only linear interactions (addition and element-wise multiplication). This "Constant Error Carousel" (CEC) allows gradients to stay healthy over hundreds of timestamps. The information flows unchanged until a gate explicitly decides to interrupt it.
- **Hidden State (h_t)**: This is a filtered, task-specific version of the cell state used for **current predictions (Short term memory)**



LSTM: Inside the LSTM Cell



The Forget Gate



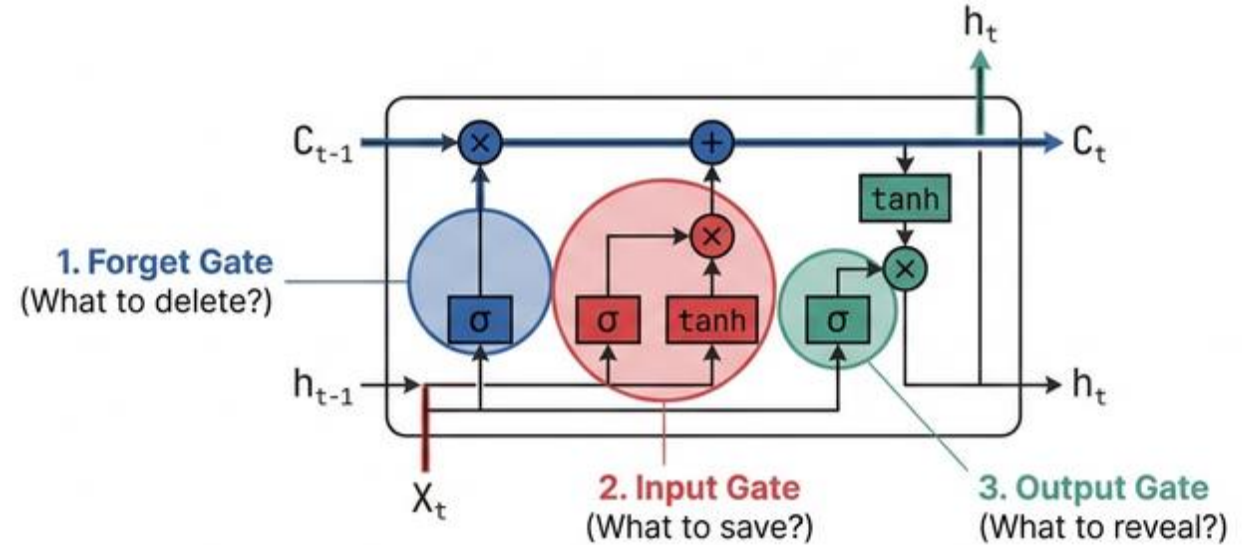
Decides what information is no longer needed?

Mechanism: Sigmoid outputs a number between 0 (Forget) and 1 (Keep).

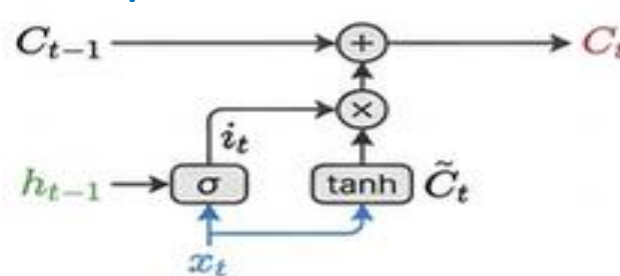
Example: If the network sees a new subject 'The dog', it uses the Forget Gate to remove the gender/number info of the old subject 'The fox'

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

The Three Gates



The Input Gate



Decides what new information is relevant enough to store in cell state

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

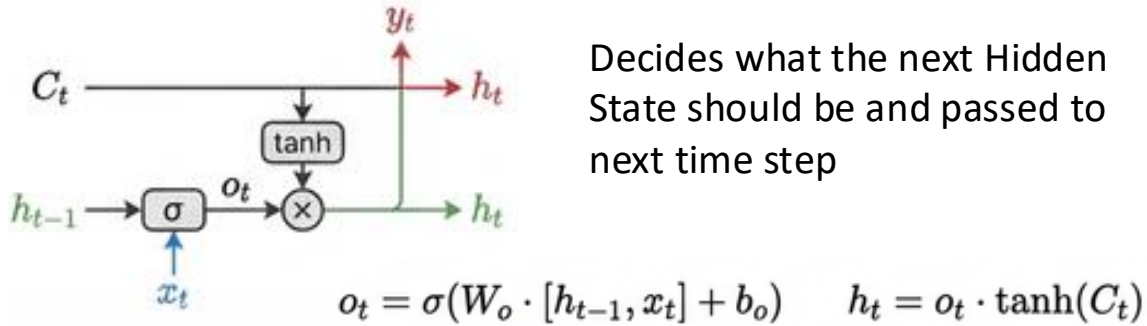
Mechanism: Two step Mechanism

- 1) Selection (Sigmoid): Decides what new info to store
- 2) Creation (Tanh): Creates new vector of new candidate values that could be added to the state

Example: The network encounter the word 'quick' (adjective). The Input Gate decides this is important context for the upcoming noun and adds it to the memory.

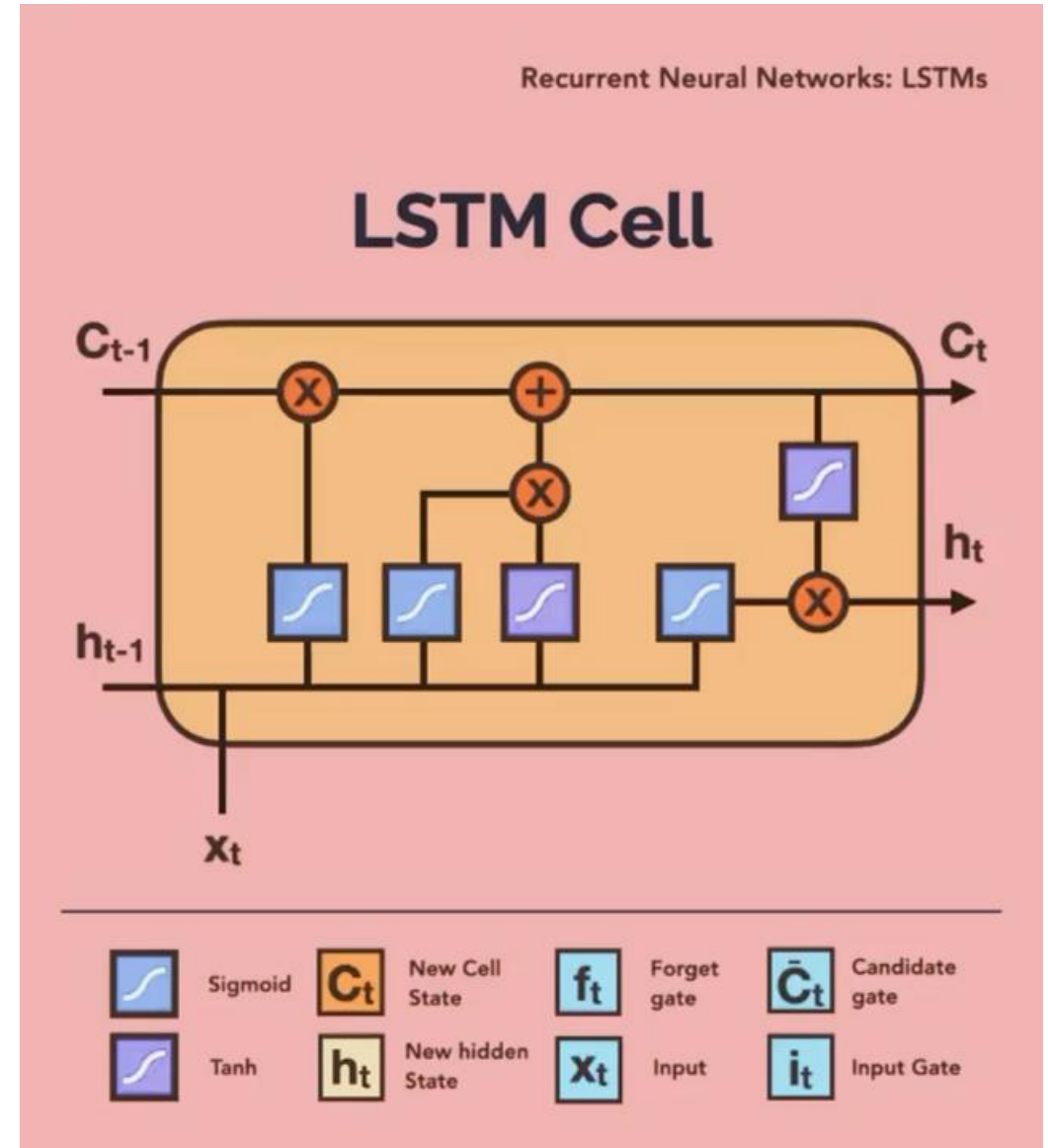
LSTM: Inside the LSTM Cell

The Output Gate



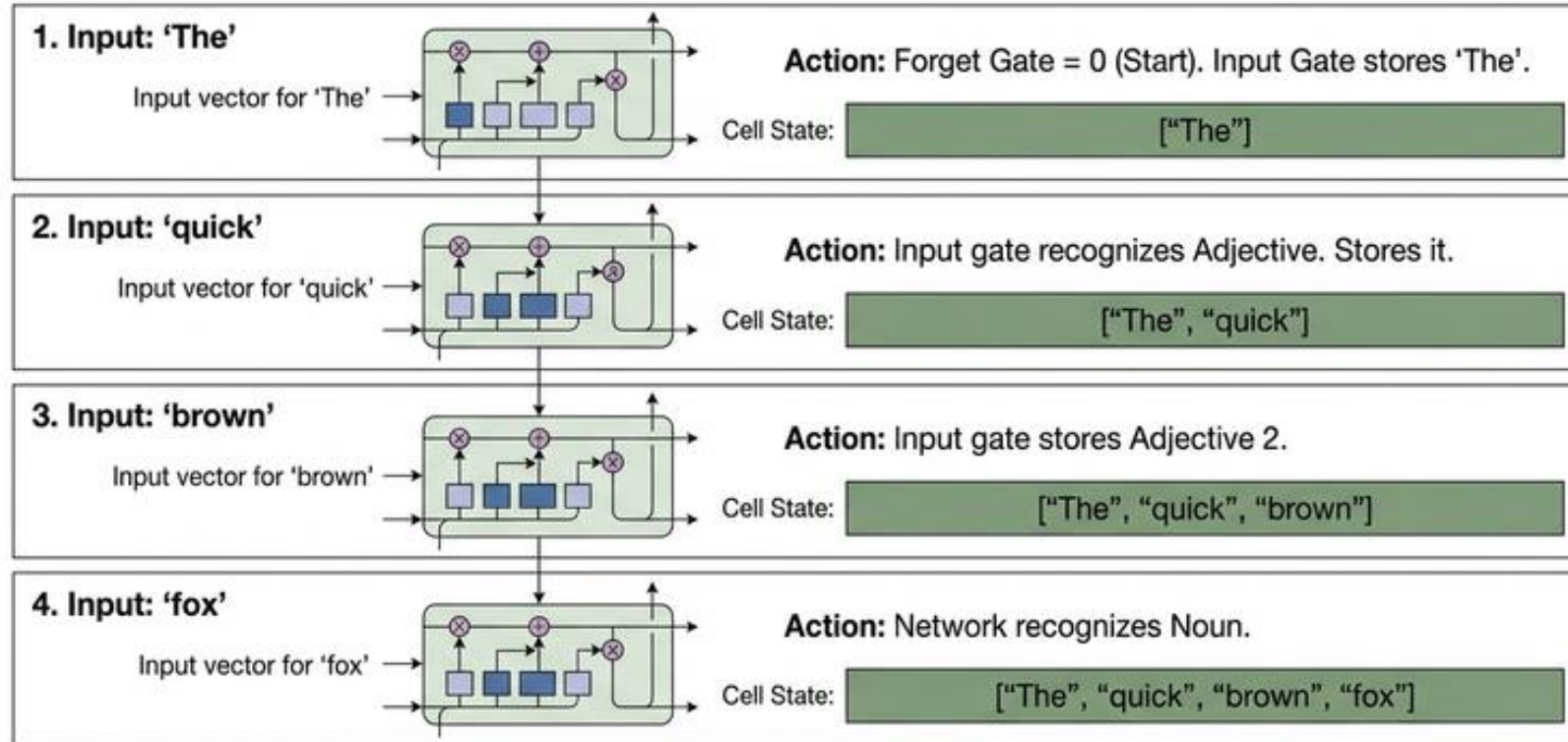
The Mechanism:

- 1) We run a Sigmoid layer to decide what parts of the cell state we are going to output.
- 2) We push the cell state through Tanh (to push values between -1 and 1)
- 3) We multiply the two. We only output the parts we decided to



LSTM: Processing Sequence

"The quick brown fox..."



By Step 4, the Cell State effectively 'remembers' the entire noun phrase despite the distance, enabling the correct prediction of the singular verb 'jumps'.

Step-by-Step Walkthrough

Timestamp 1: "The" — Initializing the State

- 1) **Forget Gate (f_1)**: Since this is the first word, the model effectively "opens" the gate to allow the start-of-sentence context to be established
- 2) **Input Gate (i_1)**: Recognizes "The" as a significant marker and writes its embedding features into the Cell State (C_1)

Timestamp 2: "quick" — Targeted Storage

- 1) **Forget Gate (f_2)**: Decides if the "start" marker from $t=1$ is still relevant.
- 2) **Input Gate (i_2)**: Identifies "quick" as an adjective. It generates a Candidate State ($\tilde{C}_2$) representing the "concept of speed."
- 3) **The Linear Update:**

$$C_2 = f_2 \odot C_1 + i_2 \odot \tilde{C}_2$$

Note: The addition (+) here is the critical "bypass" that prevents vanishing gradients.

Processing "dog" — Contextual Understanding

By the time the model reaches "dog," the Cell State contains the history of the subject ("fox") and the action ("jumps").

LSTM: Power of Node-Specific Gating

The Power of Node-Specific Gating

This means that at any given timestamp, the network has independent memory units that can be configured differently:

Node 12 might have a Forget Gate value of , allowing it to track a "Subject Number" feature across an entire paragraph.

Node 45 might have a Forget Gate value of and an Input Gate of , allowing it to "reset" and capture the meaning of a local preposition.

Node 88 might track "Negative Sentiment," only updating when specific valence-heavy words are processed.

This multi-node independence is what enables the **Constant Error Carousel (CEC)**.